

list_operator

LOGGER module-attribute

```
LOGGER = getLogger(__name__)
```

ListContainsFeatureJava module-attribute

```
ListContainsFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.ListContainsFeature')
```

ListGetFeatureJava module-attribute

```
ListGetFeatureJava = autoclass(DS_API_CLASSPATH + 'plan.features.ListGetFeature')
```

ListOperator

```
ListOperator(pyjnius_java)
```

Bases: [BaseFeaturesOperator](#)

Represents a List Operator in Pulse.

subclasses class-attribute instance-attribute

```
subclasses: List[JavaWrapper] = []
```

add_feature

```
add_feature(feature: BaseFeature, logger: bool = True) -> BaseFeature
```

Adds a new Feature to this Operator.

Parameters:

Name	Type	Description	Default
feature 	BaseFeature	New Feature.	<i>required</i>
logger 	bool	Whether to log info message or not.	True

Returns:

Type	Description
BaseFeature	New Feature.

diff

```
diff(other: BaseFeaturesOperator, only_diff: bool = True)
```

Display the differences between the features from this Operator and another Operator.

Features are compared based on their feature name. The order of the features in the Operator is not taken into account.

Parameters:

Name	Type	Description	Default
<code>other</code> ¶	BaseFeaturesOperator	BaseFeaturesOperator to compare against.	<i>required</i>
<code>only_diff</code> ¶	<code>bool</code>	Show features with differences only. Defaults to 'true'.	<code>True</code>

Raises:

Type	Description
<code>NotImplementedError</code>	When not called from a Jupyter notebook.

`get_dependencies` [¶](#)

```
get_dependencies() -> Dict[FeatureUID, Set[str]]
```

`get_dependencies_by_type` [¶](#)

```
get_dependencies_by_type() -> Dict[FeatureUID, Dict[str, list[str]]]
```

`get_feature` [¶](#)

```
get_feature(name: str) -> ListFeature
```

Gets a Feature by a given name.

Parameters:

Name	Type	Description	Default
<code>name</code> ¶	<code>str</code>	Name of the feature	<i>required</i>

Returns:

Type	Description
ListFeature	ListFeature

`get_features` [¶](#)

```
get_features() -> List[ListFeature]
```

Gets all the features in this Operator instance.

Returns:

Type	Description
<code>list of ListFeature</code>	List of ListFeature.

get_type staticmethod [↗](#)

```
get_type() -> str
```

remove_feature [↗](#)

```
remove_feature(name: str) -> bool
```

Removes the features that match the name.

Parameters:

Name	Type	Description	Default
<code>name</code> ↗	<code>str</code>	Name of the feature to remove.	<i>required</i>

Returns:

Type	Description
<code>bool</code>	If the feature is removed.

rename_feature [↗](#)

```
rename_feature(old_name: str, new_name: str)
```

to_csv [↗](#)

```
to_csv(path: Union[Path, str])
```

Exports all the features in the operator as a csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	<code>Path</code> or <code>str</code>	Path to where the csv will be written.	<i>required</i>

to_dict [↗](#)

```
to_dict() -> List[Dict]
```

to_pandas [↗](#)

```
to_pandas() -> DataFrame
```

Exports all the features in the operator as a pandas dataframe.

Returns:

Type	Description
<code>DataFrame</code>	Pandas dataframe containing features and all their editable attributes.

update_from_csv [↗](#)

```
update_from_csv(path: Union[Path, str])
```

Updates this Operator with the configuration from the csv file.

Parameters:

Name	Type	Description	Default
<code>path</code> ↗	Path or str	Path from where the csv will be read.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.

`update_from_dict` [↗](#)

```
update_from_dict(dict_feats: List[Dict])
```

`update_from_pandas` [↗](#)

```
update_from_pandas(pd_df: DataFrame)
```

Updates this Operator with the configuration from the pandas dataframe.

Parameters:

Name	Type	Description	Default
<code>pd_df</code> ↗	DataFrame	Pandas dataframe with the operator features.	<i>required</i>

See Also

Plan.from_csv : For guidelines to input csv data in the right format.